

M19 TEACHING BOOK

Interview, Technical English & Job Launch

THIS BOOK IS YOUR TEACHER

- New interview/job-launch patterns are taught from first principles.
- External videos are not required by default in practice-led M19.
- First attempts are protected from answer leakage.
- Changed retests prove transfer, not recognition.
- Every public/technical claim must match evidence.

This book is your teacher

How M19 differs from normal study

You already learned the technical stack. M19 trains retrieval, explanation, targeting and interview transfer. Therefore each lesson teaches a framework, lets you follow once, then closes support for the independent attempt.

Protection rule

Do not open Review/reference directions before saving the matching independent attempt. A correct answer seen too early can create familiarity without retrieval skill.

Failure sequence

saved attempt -> targeted teaching repair -> changed micro-proof -> validation -> reassessment. Keep original evidence append-only.

Contents / Index

DE19-01 Truthful one-page Data Engineer resume	6
DE19-02 GitHub and project portfolio final polish	9
DE19-03 LinkedIn/professional profile without overclaiming	12
DE19-04 Degree-aware job targeting: apply/skip rules	15
DE19-05 SQL interview drills	18
DE19-06 Python interview drills	21
DE19-07 Data-engineering concepts and troubleshooting drills	24
DE19-08 Project walkthrough interview	27
DE19-09 Architecture/system-design defense for junior roles	30
DE19-10 Behavioral answers and career-transition story	33
DE19-11 Technical English: explain code, failures and decisions clearly	36
DE19-12 Application tracker, feedback loop and weekly repair cycle	39
DE19-13 Timed coding test: arrays, strings and hash-map problems	42
DE19-14 Combined SQL + Python interview simulation	45
Mastery checklist	48
Revision / quick reference	49
G07 handoff	50

Module map

Job materials & targeting

DE19-01 resume -> 02 GitHub -> 03 profile -> 04 degree-aware targeting

Technical retrieval

DE19-05 SQL -> 06 Python -> 07 troubleshooting

Defense & communication

DE19-08 project walkthrough -> 09 system design -> 10 behavioral -> 11 technical English

Job-search feedback

DE19-12 application tracker/funnel

Timed synthesis

DE19-13 coding test -> DE19-14 mixed simulation -> module practical -> G07

First-use tool + route map

Files you will CREATE

Resume/profile/story drafts, SQL/Python answers, architecture notes, recordings, tracker updates and evidence attempts are learner-created. Protected reference files are never your submission.

Files you will FILL

Use supplied CSV trackers/maps/logs where named. Preserve source headers; add your own rows/results.

Evidence command

`python 07\_EVIDENCE/check\_m19\_evidence.py <M19\_ROOT> <LESSON\_ID>` checks structure only. NO_ATTEMPT_SAVED is correct until you genuinely begin.

Exact route

Book lesson -> Guided once -> Independent -> attempt.json -> validate -> Review only if needed -> Fresh Retest -> append evidence -> next lesson.

DE19-01 Truthful one-page Data Engineer resume

1. Why this matters

A recruiter sees a small amount of text before they see your code. The resume must make your strongest real evidence easy to find without turning study or project work into fake employment.

2. What you are learning

Turn M18 evidence into a one-page resume whose technical bullets are factual, narrow and defensible.

3. Prerequisite / start state

M18 portfolio/evidence complete enough to use as source material. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Imagine every resume bullet has an invisible string tied to a real file, log, query, diagram or validation result. If you pull the string and nothing real is attached, the bullet is not ready.

5. New words before we use them

evidence-backed claim: A statement supported by a real artifact you can open. | action-system-validation bullet: What you built/did, where, and how you proved it. | executed/measured flag: Internal note showing whether a claim came from real execution, static design or simulation. | overclaim: A statement stronger than the evidence supports.

6. Useful video/source

Primary: your sealed M18 evidence + GUIDED_DE19-01.md. External video: NO VIDEO BY DESIGN because this lesson tests evidence translation, not recognition.

7. Follow with me once

Teacher example only: take an inflated bullet such as "Built enterprise-scale pipelines". Narrow it to the exact project, technology and proof you actually have. Keep internal evidence tags until the claim survives checking.

Teacher model: Evidence -> action -> system -> validation/result.

8. Expected result

A truthful one-page draft. Each technical bullet has an internal evidence path and executed/measured status before the internal tags are removed.

9. Why it worked

The claim is specific enough to verify and modest enough to defend. A recruiter can ask "show me" and you know exactly what to open.

10. Common beginner mistakes

- Inventing employment or years of experience.
- Claiming production scale, users, revenue or cloud runtime not actually observed.
- Listing every tool without evidence.
- Using vague bullets such as "worked on data pipelines".

11. Stuck/recovery help

For each questionable bullet ask: What exact artifact proves this? What did I personally do? What was actually executed? If any answer is unclear, narrow or remove the claim.

12. Small independent task

Use your own M18 evidence and `resume\_evidence\_map.csv` to draft the resume. Do not copy the teacher example as your final wording.

13. Validate

Every technical claim maps to evidence; no fake employment/degree/certification; strongest bullets can be defended in under 60 seconds.

14. Teach it back

Explain why a truthful but narrower bullet is stronger than a bigger claim you cannot prove.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 3. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-01.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: learner-created one-page resume + completed internal evidence map. Evidence path: `07\_EVIDENCE/DE19-01/attempt.json` (create from template only when genuine work starts). Practice asset: `resume\_evidence\_map.csv`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-02`.

DE19-02 GitHub and project portfolio final polish

1. Why this matters

A portfolio is useful only if another person can understand and reproduce the work. Broken setup or decorative screenshots weaken strong engineering.

2. What you are learning

Audit P01, P02, P03 and Capstone as recruiter-facing, reproducible engineering evidence.

3. Prerequisite / start state

DE19-01 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Think of the repository as a small museum. The entrance sign is the README, the floor plan is the architecture, exhibits are code/evidence, and emergency exits are failure/recovery notes.

5. New words before we use them

reproducible: Another person can follow stated prerequisites and recreate the result. | pinned repository: A repo deliberately surfaced as a primary portfolio item. | secret hygiene: Credentials/tokens are absent from public history and files. | limitation: A boundary you state explicitly instead of hiding.

6. Useful video/source

Primary: M18 publication/defense material + GUIDED_DE19-02.md. External video: NO VIDEO BY DESIGN; use your own repos as the learning object.

7. Follow with me once

Teacher example only: audit a sample repo and mark one missing setup step, one weak evidence link and one secret-risk item. Repair the navigation, not the project with fake screenshots.

Teacher model: Pinned repo -> README -> architecture -> setup -> test/evidence -> failure/recovery -> limitations.

8. Expected result

A completed portfolio audit with concrete repairs and a clear order for which repos should be public/pinned.

9. Why it worked

The audit tests reproducibility, evidence and safety - the things a reviewer actually needs - rather than visual polish alone.

10. Common beginner mistakes

- Copied tutorial repo presented as original.
- README cannot reproduce setup.
- Secrets or private data committed.
- Claims exceed evidence.
- Dead or confusing files left in the public path.

11. Stuck/recovery help

Clone or copy the project to a clean folder. Follow only the README. Every place you get stuck becomes a specific documentation/setup repair.

12. Small independent task

Audit P01/P02/P03/CAP using `project\_portfolio\_audit.csv`; repair failed items before calling a repo portfolio-ready.

13. Validate

README route works from clean state; architecture and evidence are findable; no secret/PII leakage; claims match real runtime/static truth.

14. Teach it back

Explain why a beautiful repo can still be weak evidence and name the first three checks you would run.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 4. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-02.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: completed project portfolio audit + repaired repo/readme evidence. Evidence path: `07\_EVIDENCE/DE19-02/attempt.json` (create from template only when genuine work starts). Practice asset: `project\_portfolio\_audit.csv`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-03`.

DE19-03 LinkedIn/professional profile without overclaiming

1. Why this matters

A public profile helps recruiters understand your direction, but inflated titles or invented experience can damage trust immediately.

2. What you are learning

Write a clear junior/associate Data Engineer profile tied to your real stack and strongest portfolio proof.

3. Prerequisite / start state

DE19-02 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Your profile is a signpost, not a biography. It should answer: what role are you targeting, what can you defend, where is the proof, and what are you looking for?

5. New words before we use them

headline: Short role/skill positioning line visible in search. | About section: Brief narrative of direction, strengths and evidence. | Featured evidence: Public project or artifact deliberately surfaced. | availability: Truthful note about role/location/working arrangement you can accept.

6. Useful video/source

Primary: GUIDED_DE19-03.md + your M18 portfolio. External video: NO VIDEO BY DESIGN; the task is truthful writing and evidence selection.

7. Follow with me once

Teacher example only: replace an inflated title such as "Data Engineer | 3 years" with role-target wording that does not imply employment you did not have.

Teacher model: Target role + defensible stack + strongest proof + current availability.

8. Expected result

A headline, About draft and Featured plan that are coherent, specific and defensible.

9. Why it worked

The profile communicates direction without converting training into fake job history.

10. Common beginner mistakes

- Using an employer/title you do not hold.
- Inventing years of experience.
- Keyword stuffing every tool.
- Generic "passionate learner" language with no proof.

11. Stuck/recovery help

Underline every claim. If you cannot point to project evidence or a real fact about yourself, rewrite it more narrowly.

12. Small independent task

Create your profile draft in `professional\_profile\_draft\_template.md`; tie the Featured section to strongest M18 evidence.

13. Validate

No invented title/employment/years; stack names match demonstrated work; strongest proof is one click away; wording can be spoken naturally.

14. Teach it back

Explain the difference between "aspiring/targeting Data Engineer" positioning and claiming you already hold that job.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 5. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-03.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: headline + About + Featured plan. Evidence path: `07\_EVIDENCE/DE19-03/attempt.json` (create from template only when genuine work starts). Practice asset: `professional\_profile\_draft\_template.md`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-04`.

DE19-04 Degree-aware job targeting: apply/skip rules

1. Why this matters

A focused application strategy saves time and prevents mandatory eligibility rules from being confused with trainable skill gaps.

2. What you are learning

Classify roles as APPLY, MAYBE or SKIP by reading required vs preferred degree, experience and core stack literally.

3. Prerequisite / start state

DE19-03 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Treat a job description like a filter with hard gates and soft preferences. A preferred gap is a yellow light; a mandatory legal/degree/experience condition can be a red light.

5. New words before we use them

required: Condition stated as mandatory. | preferred: Helpful but not necessarily mandatory. | hard gate: Requirement that clearly excludes the application if unmet. | trainable gap: Skill gap that can reasonably be learned or demonstrated.

6. Useful video/source

Primary: GUIDED_DE19-04.md + the exact job description being evaluated. External video: NO VIDEO BY DESIGN because live postings are the source of truth.

7. Follow with me once

Teacher example only: compare two postings with identical stacks where one says "degree preferred" and the other says "B.Tech required". The classification can change even though the technologies do not.

Teacher model: APPLY when core work matches and hard requirements are met; MAYBE when gaps are preferred/trainable; SKIP when a clear hard gate excludes you.

8. Expected result

A repeatable classification method and a tracker that records why each role was applied to, held for review or skipped.

9. Why it worked

It separates actual eligibility from anxiety and prevents mass-applying to obviously mismatched senior/mandatory roles.

10. Common beginner mistakes

- Treating every preferred item as mandatory.
- Ignoring an explicit mandatory qualification.
- Applying to senior roles because the stack looks familiar.
- Assuming one rejection proves the whole field is closed.

11. Stuck/recovery help

Read the posting twice: first only hard requirements, then preferred skills. Mark each line REQUIRED/PREFERRED/UNCLEAR before deciding.

12. Small independent task

Classify a set of roles in `job\_targeting\_tracker.csv`; document the exact apply/skip reason rather than "good/bad fit".

13. Validate

Each decision quotes/paraphrases a requirement; no invented eligibility; core job work matches your demonstrated stack for APPLY roles.

14. Teach it back

Explain why a missing preferred skill does not automatically mean SKIP and give one example of a true hard gate.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 6. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-04.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: degree-aware job-targeting tracker with APPLY/MAYBE/SKIP rationale. Evidence path: `07\_EVIDENCE/DE19-04/attempt.json` (create from template only when genuine work starts). Practice asset: `job\_targeting\_tracker.csv`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-05`.

DE19-05 SQL interview drills

1. Why this matters

SQL interviews test whether you can translate a business question into the right grain and defend correctness under time pressure.

2. What you are learning

Solve timed SQL questions while stating grain, controlling joins/windows and validating independently.

3. Prerequisite / start state

DE19-04 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Think of each query as a small pipeline: business question enters, row grain changes through stages, and a validation checkpoint guards the output.

5. New words before we use them

output grain: What one row in the result represents. | deterministic ranking: Tie-breaking rules make repeated ranking stable. | join multiplication: One-to-many joins create extra rows and may inflate totals. | independent validation: A second check not merely repeating the same query logic.

6. Useful video/source

Primary: packaged SQL mock + GUIDED_DE19-05.md. Refresh only weak concepts from earlier SQL module; do not watch a solution immediately before the timed attempt.

7. Follow with me once

Teacher example only: before typing a top-2-per-region query, say "one row per selected order" and choose a deterministic tie-breaker.
Teacher model: Question -> state grain -> write query -> inspect duplicates/nulls -> independent count/total -> explain trade-off.

8. Expected result

Timed SQL answers plus a one-line grain statement and at least one independent validation for each important result.

9. Why it worked

The method catches business-wrong queries that happen to run successfully.

10. Common beginner mistakes

- Starting syntax before stating grain.
- Using DISTINCT to hide join multiplication.
- Ranking without deterministic tie-breaker.
- No validation because the result "looks right".

11. Stuck/recovery help

Stop coding. Restate the question and grain. Inspect source keys. Build the query in named stages; run row-count/total checks after each grain change.

12. Small independent task

Complete `sql\_mock\_questions.md` against `sql\_mock\_setup.sql` closed-book; save SQL, outputs and validation notes.

13. Validate

Queries answer intended grain; join/window behavior is explainable; important counts/totals reconcile; tie handling is deterministic.

14. Teach it back

Explain one query from business question to grain to validation without reading the SQL line by line.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 7. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-05.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: timed SQL solution file + validation evidence. Evidence path: `07\_EVIDENCE/DE19-05/attempt.json` (create from template only when genuine work starts). Practice asset: `sql\_mock\_questions.md + sql\_mock\_setup.sql`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-06`.

DE19-06 Python interview drills

1. Why this matters

Junior DE Python work is usually judged by clear data handling, explicit failures, testable behavior and your ability to explain the code.

2. What you are learning

Implement small data functions from a contract, handle bad input explicitly and test edge cases.

3. Prerequisite / start state

DE19-05 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Treat a function like a checkpoint gate: input enters under a contract, valid rows take the normal path, bad input takes an explicit failure path, and output has a deterministic shape.

5. New words before we use them

input contract: Expected fields/types/behavior before the function runs. | deterministic: Same input and state produce the same defined output. | edge case: Less-common but valid/invalid condition that tests assumptions. | exception scope: The smallest code region where an error should be caught/handled.

6. Useful video/source

Primary: packaged Python mock + GUIDED_DE19-06.md. Refresh only the exact weak Python concept after the first attempt.

7. Follow with me once

Teacher example only: parse three tiny records, deliberately include one malformed numeric, and choose whether to reject/quarantine rather than silently converting it.

Teacher model: Input contract -> transform -> explicit failure path -> deterministic output -> edge-case tests -> practical complexity.

8. Expected result

Readable functions with explicit invalid-input behavior and small tests for happy, malformed, duplicate and empty cases where relevant.

9. Why it worked

The code becomes explainable and failures are visible instead of silently corrupting data.

10. Common beginner mistakes

- Catching every exception.
- Silently dropping bad rows.
- Hidden mutable state.
- Code works only for the happy path.
- Cannot explain why a data structure was chosen.

11. Stuck/recovery help

Write the input/output contract in plain English first. Add one tiny failing example. Fix the smallest behavior, then rerun previous passing cases.

12. Small independent task

Complete ``python_mock.py`` from blank implementations. Add your own small assertions/examples and save first failure before repair.

13. Validate

Functions behave deterministically; invalid values are handled explicitly; edge cases are tested; you can explain code and practical complexity.

14. Teach it back

Explain one function as contract -> transformation -> failure path -> checks, not as a line-by-line recital.

15. Protected review + changed retry

Save the first attempt first. If weak, open ``05_PROTECTED_REVIEW/REVIEW_AFTER_ATTEMPT_M19.pdf`` page 8. Diagnose the smallest weak dimension, close Review, then open ``06_FRESH_RETEST/RETEST_DE19-06.md`` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: completed Python mock + test/output evidence. Evidence path: ``07_EVIDENCE/DE19-06/attempt.json`` (create from template only when genuine work starts). Practice asset: ``python_mock.py``. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: ``DE19-07``.

DE19-07 Data-engineering concepts and troubleshooting drills

1. Why this matters

Interviewers often care less about naming tools than whether you can investigate a broken system safely without destroying evidence.

2. What you are learning

Use a repeatable troubleshooting ladder across orchestration, data quality, incremental state, Spark, streaming, lakehouse, security and SQL incidents.

3. Prerequisite / start state

DE19-06 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Imagine a dark building with an alarm. Do not demolish the building. First locate the affected floor, check the nearest sensor, run a safe test, repair one cause, then verify the alarm and neighboring systems.

5. New words before we use them

blast radius: Which users/data/stages are affected. | first evidence: Logs, metrics, row counts or state inspected before changing anything. | safe test: Small diagnostic action that does not destroy evidence or widen impact. | reconciliation: Independent check that repaired output matches trusted controls.

6. Useful video/source

Primary: `troubleshooting\_drills.md` + GUIDED_DE19-07.md; use M16/M17 quick references only after first attempt identifies a weak concept.

7. Follow with me once

Teacher example only: DAG is green but mart total is low. Check source arrival/stage counts/filter boundaries before clearing state or rerunning everything.

Teacher model: Symptom -> blast radius -> first evidence -> likely layer -> smallest safe test -> repair -> rerun boundary -> reconciliation -> prevention.

8. Expected result

Structured incident answers that name evidence, hypothesis, safe diagnostic, repair boundary, validation and prevention.

9. Why it worked

The ladder separates observation from assumption and protects persisted state while narrowing the cause.

10. Common beginner mistakes

- Restarting everything first.
- Deleting state/watermarks before diagnosis.
- Naming a root cause with no evidence.
- Fixing symptom but not reconciling output.

11. Stuck/recovery help

If stuck, name the last known-good boundary and the first bad boundary. Ask what evidence can distinguish source, transform, state, orchestration or serving failure.

12. Small independent task

Work through `troubleshooting\_drills.md`; write the first three evidence checks before proposing any destructive action.

13. Validate

Diagnosis starts from evidence; safe test is bounded; repair matches likely cause; rerun boundary is explicit; output is reconciled; prevention is plausible.

14. Teach it back

Walk through one incident and clearly separate symptom, evidence, hypothesis, test, repair and proof.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 9. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-07.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: incident diagnosis notes for all drills + one changed retry. Evidence path: `07\_EVIDENCE/DE19-07/attempt.json` (create from template only when genuine work starts). Practice asset: `troubleshooting\_drills.md`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-08`.

DE19-08 Project walkthrough interview

1. Why this matters

Your portfolio becomes interview evidence only when you can explain the problem, design, controls, failure recovery and limitations without reading the README.

2. What you are learning

Deliver a structured walkthrough of an M18 project and handle skeptical follow-up questions.

3. Prerequisite / start state

DE19-07 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Imagine guiding a new engineer through the system from left to right: business need, sources, architecture, trusted output, how it fails, how you know, and what you would change next.

5. New words before we use them

walkthrough: Structured verbal tour of a project from problem to evidence. | trade-off: Benefit accepted with a known cost/limitation. | 10x change: Reasoning about how design might change under much larger scale or tighter SLA. | runtime truth: Clear statement of what was actually executed versus static/simulated.

6. Useful video/source

Primary: your M18 project + `project\_defense\_questions.md` + GUIDED_DE19-08.md. No external video is needed; your own project is the source.

7. Follow with me once

Teacher example only: give a 90-second mini walkthrough of one architecture diagram, then answer "what can fail?" with evidence and recovery rather than buzzwords.

Teacher model: Problem -> sources -> architecture -> grain -> quality -> idempotency -> orchestration -> failure/recovery -> security -> limitations -> 10x change.

8. Expected result

A concise project defense with evidence pointers and honest limitations, followed by changed-project retest.

9. Why it worked

The structure keeps the answer technical and evidence-led instead of turning into a feature list.

10. Common beginner mistakes

- Reading README aloud.
- Reciting technology names without decisions.
- Calling a project production-ready without runtime evidence.
- No failure/recovery story.

11. Stuck/recovery help

Return to the M18 evidence index. For each section, choose one concrete artifact you can show. If you cannot show it, narrow the claim.

12. Small independent task

Use `project\_defense\_questions.md` on your strongest M18 project; record or write answers without the project book open.

13. Validate

All 10 defense categories are covered; technical claims point to evidence; one failure/recovery and one limitation are explicit; runtime truth is honest.

14. Teach it back

Explain why your chosen architecture was reasonable for the actual requirements and name one condition that would justify redesign.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 10. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-08.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: project walkthrough recording/notes + evidence pointers. Evidence path: `07\_EVIDENCE/DE19-08/attempt.json` (create from template only when genuine work starts). Practice asset: `project\_defense\_questions.md`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-09`.

DE19-09 Architecture/system-design defense for junior roles

1. Why this matters

Junior system-design interviews test whether you can start from requirements and choose the simplest defensible architecture, not whether you can draw the most boxes.

2. What you are learning

Turn workload/latency/correctness/security/cost requirements into an architecture and defend trade-offs.

3. Prerequisite / start state

DE19-08 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Think of architecture as a chain of decisions. Every box should have an incoming requirement and an outgoing responsibility. If you cannot explain why a box exists, remove or question it.

5. New words before we use them

SLA: Required service timing/availability target. | source contract: Expected source grain/schema/delivery behavior. | operability: How easily the system can be monitored, recovered and maintained. | revisit trigger: Requirement change that would justify a different design.

6. Useful video/source

Primary: M17 architecture frameworks + packaged scenario + GUIDED_DE19-09.md. External solution video is intentionally avoided before the attempt.

7. Follow with me once

Teacher example only: start batch-first for a daily reporting SLA; add streaming only when the urgent-event SLA actually requires it.
Teacher model: Requirements -> scale/latency -> source contracts -> storage/compute -> orchestration -> correctness -> security -> observability -> failure modes -> cost/operability.

8. Expected result

A requirements-led diagram/decision narrative with failure modes, controls, trade-offs and revisit triggers.

9. Why it worked

It prevents tool-first architecture and makes complexity proportional to a requirement.

10. Common beginner mistakes

- Kafka/Spark/Kubernetes by default.
- No data grain or state model.
- No recovery/observability.
- Ignoring cost/operability.
- No reason for batch vs streaming choice.

11. Stuck/recovery help

Erase tool names temporarily. Write five requirements and three failure risks. Re-add only components that satisfy a stated need.

12. Small independent task

Solve `system\_design\_scenario.md` from blank notes; then defend the changed ClinicBridge retest scenario without copying the first boxes.

13. Validate

Every major component maps to a requirement; batch/stream split is justified; correctness/security/recovery are covered; at least one simpler alternative is discussed.

14. Teach it back

Explain one component you deliberately did NOT choose and the requirement that would make you add it later.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 11. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-09.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: architecture sketch/ADR-style notes + defense recording. Evidence path: `07\_EVIDENCE/DE19-09/attempt.json` (create from template only when genuine work starts). Practice asset: `system\_design\_scenario.md`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-10`.

DE19-10 Behavioral answers and career-transition story

1. Why this matters

Behavioral interviews test judgment, ownership and learning. You can answer truthfully from course/project/life evidence without inventing a workplace story.

2. What you are learning

Build concise, factual examples for failure, ambiguity, feedback, learning and career transition.

3. Prerequisite / start state

DE19-09 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Treat each story like a short incident report: context sets the scene, responsibility says what was yours, action shows decisions, result shows outcome, verification proves it, reflection shows what changed.

5. New words before we use them

behavioral prompt: Question about how you acted in a situation. | ownership: What responsibility/decision was genuinely yours. | verification: Observable result or evidence beyond "it went well". | reflection: What you learned or would change next time.

6. Useful video/source

Primary: GUIDED_DE19-10.md + your real course/project/life evidence. NO VIDEO BY DESIGN; authenticity matters more than copying a polished answer.

7. Follow with me once

Teacher example only: turn "I struggled with SQL" into a specific learning/repair story with a failed query, diagnosis, changed practice and later proof - without claiming employment.

Teacher model: Context -> responsibility -> action -> result -> verification -> reflection.

8. Expected result

A small story bank you can deliver naturally, including a truthful non-tech-to-DE transition explanation.

9. Why it worked

The structure makes evidence and decisions visible while keeping the story honest and concise.

10. Common beginner mistakes

- Invented workplace stories.
- Blaming another person/tool.
- Perfect story with no real difficulty.
- No result/verification.
- Memorized paragraph that sounds unnatural.

11. Stuck/recovery help

Choose a smaller real event. Write only six one-line fields using the model. Then speak from those anchors instead of memorizing prose.

12. Small independent task

Use `behavioral\_prompts.md` and `behavioral\_story\_bank\_template.md` to prepare at least five distinct truthful stories.

13. Validate

Stories are factual; responsibility is clear; action contains decisions; result/verification is concrete; reflection is specific; no invented employment.

14. Teach it back

Deliver your career-transition answer in 60-90 seconds and explain why project evidence is not the same as professional employment.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 12. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-10.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: behavioral story bank + transition-story recording/notes. Evidence path: `07\_EVIDENCE/DE19-10/attempt.json` (create from template only when genuine work starts). Practice asset: `behavioral\_prompts.md + behavioral\_story\_bank\_template.md`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-11`.

DE19-11 Technical English: explain code, failures and decisions clearly

1. Why this matters

Interview clarity is not about accent. It is about giving the listener a structure they can follow and correcting yourself calmly when wording is imperfect.

2. What you are learning

Explain SQL, Python, pipeline failures and design decisions in plain technical English.

3. Prerequisite / start state

DE19-10 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Imagine handing the listener five labeled boxes: what it is, what changed, why, how you checked, and how it can fail. Put one sentence in each box.

5. New words before we use them

jargon-first: Starting with dense terms before giving meaning/context. | self-correction: Calmly correcting wording without panic or hiding uncertainty. | technical noun: The exact object: table, row, watermark, partition, DAG, consumer, etc. | verification sentence: Simple statement of how correctness was checked.

6. Useful video/source

Primary: GUIDED_DE19-11.md + your own SQL/Python/project evidence. External video is optional and not required for mastery.

7. Follow with me once

Teacher example only: explain a LEFT JOIN in plain English before using "cardinality"; then add one validation sentence about row counts.
Teacher model: Object -> change -> why -> check -> failure risk.

8. Expected result

Short recorded explanations that are technically correct, structured and understandable without reading.

9. Why it worked

The fixed structure reduces cognitive load and lets technical correctness carry the answer even when vocabulary is simple.

10. Common beginner mistakes

- Long dictionary definitions.
- Jargon before meaning.
- Rushing after forgetting a word.
- Pretending certainty when unsure.
- No example or check.

11. Stuck/recovery help

Stop, say "Let me restate that simply," name the object, then complete the five-part model. Technical precision matters more than sophisticated vocabulary.

12. Small independent task

Use `technical\_english\_recording\_log.csv` to record several 60-120 second explanations; note one repair after listening back.

13. Validate

Listener can identify object/change/reason/check/risk; technical meaning is correct; correction is calm; no reading from script.

14. Teach it back

Explain one pipeline failure in plain English to a non-specialist, then give the more technical version.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 13. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-11.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: recordings + self-review log. Evidence path: `07\_EVIDENCE/DE19-11/attempt.json` (create from template only when genuine work starts). Practice asset: `technical\_english\_recording\_log.csv`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-12`.

DE19-12 Application tracker, feedback loop and weekly repair cycle

1. Why this matters

Job searching becomes more useful when each batch gives evidence about where the funnel is failing instead of producing random repeated applications.

2. What you are learning

Track targeted applications by stage and choose one bounded repair experiment from observed funnel patterns.

3. Prerequisite / start state

DE19-11 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Think of applications as a pipeline: application -> screen -> assessment -> technical -> final -> offer. A low conversion at one stage tells you where to investigate first.

5. New words before we use them

funnel: Ordered stages candidates move through. | conversion: Movement from one stage to the next. | bottleneck: Stage where progress repeatedly drops. | repair experiment: One specific change tested for a limited batch.

6. Useful video/source

Primary: your tracker + GUIDED_DE19-12.md. Live employer responses are the eventual runtime evidence; the static course does not fabricate them.

7. Follow with me once

Teacher example only: if screens are strong but assessments fail, changing the resume is not the first experiment; practice the assessment competency and compare the next batch.

Teacher model: Targeted role -> truthful application -> screen -> assessment -> technical -> final -> offer -> diagnose stage -> one repair experiment.

8. Expected result

An application tracker plus weekly review rule that produces one evidence-based experiment rather than many simultaneous changes.

9. Why it worked

Stage-specific evidence helps distinguish targeting/resume problems from assessment/interview problems.

10. Common beginner mistakes

- No tracker.
- Mass applying to ineligible roles.
- Changing resume, profile, projects and interview prep all at once.
- Repeated chasing with no new evidence.
- Endless course polishing without testing the market.

11. Stuck/recovery help

Summarize the last meaningful batch by stage. Pick the earliest repeated drop-off. Change one variable for the next batch and define what result would support the hypothesis.

12. Small independent task

Fill `application\_funnel.csv` and `job\_targeting\_tracker.csv` with a simulated or real batch, clearly labeling simulation if no applications have yet been sent.

13. Validate

Stage data is honest; bottleneck is derived from evidence; experiment targets that stage; success criterion is stated before the next batch.

14. Teach it back

Given two different funnels, explain why they require different repairs.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 14. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-12.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: application tracker + weekly funnel-review procedure. Evidence path: `07\_EVIDENCE/DE19-12/attempt.json` (create from template only when genuine work starts). Practice asset: `application\_funnel.csv` + `job\_targeting\_tracker.csv`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `DE19-13`.

DE19-13 Timed coding test: arrays, strings and hash-map problems

1. Why this matters

General coding screens can appear even for data-engineering roles. A small set of data-structure patterns helps you reason without memorizing hundreds of solutions.

2. What you are learning

Solve simple arrays/strings/hash-map tasks from contract to examples, implementation, edge cases and complexity.

3. Prerequisite / start state

DE19-12 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

A hash map is like labeled drawers. Instead of walking the whole room repeatedly, you can look up a key in the right drawer while scanning once.

5. New words before we use them

frequency map: Key -> count mapping. | lookup table: Mapping used for fast membership/value retrieval. | two-sum pattern: Store seen values so each new value can check its complement. | time complexity: How work grows with input size at a high level.

6. Useful video/source

Primary: packaged coding mock + GUIDED_DE19-13.md. Open protected reference solutions only after a saved first attempt.

7. Follow with me once

Teacher example only: count characters with a dictionary, then explain why one pass plus lookups is clearer than nested scans.

Teacher model: Restate contract -> examples -> choose data structure -> implement -> edge cases -> complexity -> verbal explanation.

8. Expected result

Working solutions to the packaged tasks from a blank file plus explanations of data structure and practical complexity.

9. Why it worked

The process prioritizes understanding the input/output contract and reduces solution memorization.

10. Common beginner mistakes

- Coding before clarifying output.
- Ignoring empty/duplicate/tie cases.
- Memorizing exact answers.
- Using $O(n^2)$ nested loops without noticing a simpler lookup pattern.

11. Stuck/recovery help

Use a tiny hand-worked example. State what information must be remembered while scanning. That usually reveals whether a set/dict is useful.

12. Small independent task

Complete ``coding_mock_a.py`` closed-book, then do the changed retest from a blank file.

13. Validate

Correct outputs on normal and edge cases; deterministic behavior where ties exist; complexity explanation is reasonable; code can be explained without notes.

14. Teach it back

Explain why a dictionary/set helps one problem and why it may not be needed for another.

15. Protected review + changed retry

Save the first attempt first. If weak, open ``05_PROTECTED_REVIEW/REVIEW_AFTER_ATTEMPT_M19.pdf`` page 15. Diagnose the smallest weak dimension, close Review, then open ``06_FRESH_RETEST/RETEST_DE19-13.md`` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: timed coding solutions + edge-case tests + explanation notes. Evidence path: ``07_EVIDENCE/DE19-13/attempt.json`` (create from template only when genuine work starts). Practice asset: ``coding_mock_a.py``. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: ``DE19-14``.

DE19-14 Combined SQL + Python interview simulation

1. Why this matters

Real interviews mix retrieval, coding, troubleshooting and communication. The final M19 lesson tests switching between these skills before G07.

2. What you are learning

Run a timed mixed mock, preserve the first attempt, diagnose misses and prove repair with a changed mixed retest.

3. Prerequisite / start state

DE19-13 evidence complete with no unresolved critical failure. Open only the learner-facing practice asset for this lesson. Keep protected Review/reference answers closed until after a saved first attempt.

4. Picture it in your head

Think of the mock as a flight simulator: you do not need a perfect score; you need every critical control usable, and any failure must be diagnosed and successfully retested before moving on.

5. New words before we use them

mixed mock: Timed assessment combining multiple competency types. | miss classification: Label the weak dimension: knowledge, reasoning, syntax, validation, communication, time or evidence mismatch. | changed retest: Different prompt/data requiring the same competency. | critical failure: Failure that cannot be averaged away by strength elsewhere.

6. Useful video/source

Primary: packaged combined mock + GUIDED_DE19-14.md. No solution media before the first attempt; this is retrieval practice.

7. Follow with me once

Teacher example only: after a SQL miss caused by join multiplication, repair cardinality reasoning and test it on changed tables; do not simply rerun the same answer.

Teacher model: Save first attempt -> classify misses -> protected review -> targeted repair -> close review -> changed mixed mock -> defend without notes.

8. Expected result

A complete first mixed attempt, diagnosis, targeted repairs where needed, and a changed mixed retest with no unresolved critical failure.

9. Why it worked

The changed retest distinguishes transfer from recognition after seeing feedback.

10. Common beginner mistakes

- Opening solutions during first attempt.
- Repeating the same mock after review.
- Averaging away a critical weakness.
- Calling a timed failure mastery because untimed work is strong.

11. Stuck/recovery help

Classify each miss precisely. Repair only the smallest weak competency with one short drill, then close all help and test it in changed context.

12. Small independent task

Run `combined\_mock\_b.md` under its time box, then create/use the changed DE19-14 retest without reusing first answers.

13. Validate

First evidence is preserved; each miss has a diagnosis; changed retest is genuinely different; SQL/Python/troubleshooting/project explanation are independently usable; no critical failure remains.

14. Teach it back

Explain the weakest first-attempt dimension, the repair you chose, and what changed evidence proves transfer.

15. Protected review + changed retry

Save the first attempt first. If weak, open `05\_PROTECTED\_REVIEW/REVIEW\_AFTER\_ATTEMPT\_M19.pdf` page 16. Diagnose the smallest weak dimension, close Review, then open `06\_FRESH\_RETEST/RETEST\_DE19-14.md` and solve the changed task. Preserve attempt-01.

16. Evidence/mastery + exact next route

Expected artifact: timed mixed-mock evidence + repair log + changed retest evidence. Evidence path: `07\_EVIDENCE/DE19-14/attempt.json` (create from template only when genuine work starts). Practice asset: `combined\_mock\_b.md`. Mastery means independently usable + truthful + validated + no unresolved critical failure. Next: `G07`.

End-of-module mastery checklist

Mastery item

Resume/profile claims are evidence-backed and truthful.

Mastery item

P01/P02/P03/CAP can be navigated and defended without reading.

Mastery item

SQL and Python timed tasks are independently usable and validated.

Mastery item

Troubleshooting starts with evidence and safe tests.

Mastery item

Junior architecture starts from requirements, not tools.

Mastery item

Behavioral answers use real evidence and clear reflection.

Mastery item

Technical English is structured and calm; accent is not a scoring shortcut.

Mastery item

Application targeting/funnel has an evidence-based repair loop.

Mastery item

Coding and mixed mock changed retests are complete.

Mastery item

No critical failure is unresolved.

Revision / quick reference

Claims

Evidence -> action -> system -> validation/result.

SQL

Question -> grain -> query -> duplicate/null check -> independent control.

Python

Contract -> transform -> explicit failure -> deterministic output -> tests.

Troubleshooting

Symptom -> blast radius -> evidence -> safe test -> repair -> rerun boundary -> reconcile.

Project defense

Problem -> sources -> architecture -> grain -> quality -> failure/recovery -> security -> limits -> 10x.

System design

Requirements first; add complexity only when a requirement pays for it.

Behavioral

Context -> responsibility -> action -> result -> verification -> reflection.

Technical English

Object -> change -> why -> check -> failure risk.

Failure loop

saved attempt -> targeted teaching repair -> changed micro-proof -> validation -> reassessment.

G07 handoff

Before opening G07

Finish all 14 lesson evidence routes and the M19 Module Practical. Strong performance in some areas cannot average away an unresolved critical failure in another.

What G07 is for

G07 is the final job-ready practical/simulation after M19, not a bonus attempt inside M19. Keep G07 closed until M19 evidence is complete.

What M19 does NOT prove

Static QA cannot prove a future recruiter response, live interview pass, external job eligibility or public-profile state. Those remain real-world evidence.

Canonical next route

M18 -> M19 -> G07. Production Mentor/GitHub synchronization happens only after the course-body rebuild sequence and final QA.